

A From-Scratch Guide to “Rent or Toxicity? Exploiting and Defending Learning Market Makers on Hyperliquid”

Companion to 6.S974 Wargame Deliverable 1

Prepared for the project team

September 2026

Abstract

This document explains, from zero, every idea that appears in our three-page proposal. It assumes you have never seen an order book, never heard of a market maker, and have not taken a course in game theory or reinforcement learning. Each concept is introduced with a small numerical example before any notation is used, and every symbol in the proposal is decoded in a table. Read it front to back once; afterwards use the glossary (Section 12) as a lookup.

Contents

1	The One-Paragraph Version	3
2	Part I: How a Modern Market Works	3
2.1	Prices are not a single number	3
2.2	Two kinds of orders: limit and market	4
2.3	Price–time priority	4
2.4	Tick size and lot size	4
2.5	Fees	4
3	Part II: Market Makers and Why They Get Hurt	5
3.1	What a market maker does	5
3.2	Risk 1: inventory	5
3.3	Risk 2: adverse selection — the central idea	5
3.4	The Glosten–Milgrom model: computing the fair spread	6
3.5	Belief updating after each trade	7
3.6	Markouts: measuring adverse selection from data	7
3.7	The rent-or-toxicity puzzle, stated plainly	8
4	Part III: Games, Equilibrium, and Collusion	8
4.1	What “strategic” means	8
4.2	Bertrand competition: why the competitive spread is the equilibrium	9
4.3	Repeated games and collusion	9
4.4	Price-trigger strategies and the impulse-response test	9

4.5	The collusion index Δ	10
5	Part IV: Learning Agents	10
5.1	Reinforcement learning in one paragraph	10
5.2	Q-learning, with a worked example	10
5.3	Why learning agents end up colluding (two different mechanisms)	11
5.4	No-regret (Hedge) learners	11
5.5	Deep RL (PPO) in one line	12
5.6	Best response, exploitability, and PSRO	12
6	Part V: Perpetual Futures and the Hyperliquid Exchange	12
6.1	What is a perpetual future?	12
6.2	Hyperliquid in plain words	12
6.2.1	Mechanic 1: persistent wallet identity	13
6.2.2	Mechanic 2: request budgets (rate limits)	13
6.2.3	Mechanic 3: priced latency	14
6.2.4	Mechanic 4: tick-size discontinuities	15
6.2.5	Mechanic 5: HLP, an inspiration for the fixed-policy benchmark	15
6.2.6	Mechanic 6: liquidation cascades and 10 October 2025	15
6.2.7	The dataset	15
7	Part VI: The Model, Line by Line	16
7.1	The agents	16
7.2	State, payoffs, sequence	17
7.3	Benchmarks and metrics	18
7.4	The simplifying assumptions, and why each is acceptable for now	19
8	Part VII: The Research Questions and the Attacks	19
8.1	RQ1: rent or toxicity from the tape	19
8.2	RQ2: exploitability of learned quoters	20
8.3	RQ3: robust quoter and venue design	20
8.4	How this fills the four course modules	21
9	Part VIII: The Literature, in Plain Words	21
10	Part IX: The Plan	22
10.1	The three feedback questions, explained	22
11	Part X: What We Found, and What It Changed (added October 9)	23
11.1	Five words you need, from the ground up	23
11.2	What the simulator said	24
11.3	What the data said	24
11.4	What changed in the plan	24
12	Glossary	25

1 The One-Paragraph Version

Some traders, called *market makers*, make money by constantly offering to buy a little below the “true” price and sell a little above it. The gap between their buy price and sell price is the *spread*, and it is their revenue. Their cost is that some of the people who trade with them know something they don’t, so the market maker ends up buying right before the price falls or selling right before it rises. That cost is called *adverse selection*. Increasingly, market makers are computer programs that learn by trial and error. Several recent papers found that such programs, without ever communicating, drift into charging spreads wider than competition should allow—a kind of accidental cartel. Our project asks a question those papers don’t: **if you are a smart trader arriving at such a market, can you tell whether the wide spread is a cartel you can undercut for profit, or a warning that the trading flow is dangerous—and how do you build a market maker that isn’t the one being exploited?** We study this in a simulator and on real data from Hyperliquid, a cryptocurrency exchange that publishes every single order together with the identity of who sent it.

2 Part I: How a Modern Market Works

2.1 Prices are not a single number

When people say “Bitcoin is at \$100,000,” that is a simplification. At any instant on an exchange there are really *two* prices:

- the **best bid**: the highest price anyone is currently willing to *pay* to buy;
- the **best ask** (or *offer*): the lowest price anyone is currently willing to *accept* to sell.

The best bid is always below the best ask (otherwise the two parties would just trade and both prices would vanish). The difference is the **bid–ask spread**, and the average of the two is the **mid-price**, usually written m .

Example: a tiny order book

Suppose the exchange currently holds these standing offers to trade a coin:

Bids (buyers)		Asks (sellers)	
Price	Size	Price	Size
99.8	5	100.2	3
99.7	12	100.3	8
99.5	40	100.5	25

The best bid is 99.8 (someone will buy up to 5 units at that price). The best ask is 100.2. The spread is $100.2 - 99.8 = 0.4$. The mid-price is $m = (99.8 + 100.2)/2 = 100.0$. This whole table is called the **limit order book (LOB)**. Each row is a *price level*; the size is the total quantity resting at that level, possibly from several different traders.

2.2 Two kinds of orders: limit and market

- A **limit order** says “I will buy (or sell) up to q units, but only at price p or better.” If nobody is currently willing to trade at that price, the order *rests* in the book and waits. Every row in the table above is made of resting limit orders. A trader who places a resting order is called a **maker** because they “make” liquidity—they create the option for someone else to trade.
- A **market order** says “buy (or sell) q units right now at whatever the best available price is.” It executes immediately against the resting limit orders. A trader who does this is a **taker**: they take liquidity that someone else provided.

Example: a market order eats the book

Using the book above, a taker sends a market order to *buy 10 units*. It matches against the asks, cheapest first:

- 3 units at 100.2 (that level is now empty),
- 7 units at 100.3 (that level now has $8 - 7 = 1$ unit left).

The taker paid $3 \times 100.2 + 7 \times 100.3 = 1004.7$, an average of 100.47 per unit, even though the “price” was quoted as 100.0. That extra 0.47 per unit is the cost of demanding immediacy. The new best ask is 100.3 with size 1.

2.3 Price–time priority

When several resting orders sit at the same price, which one trades first? Almost every exchange, including Hyperliquid, uses **price–time priority**: better price first; among equal prices, the order that arrived *earliest* trades first. This creates a **queue** at every price level, and *queue position* is valuable: the first order in line gets filled, the last may never be.

Intuition

Think of the order book as a set of deli-counter lines, one per price. Placing a limit order is joining a line. A market order is a customer who walks up and is served by whoever is at the front of the cheapest line. If you cancel and re-submit your order, you go to the back of the line.

2.4 Tick size and lot size

Prices on an exchange cannot be arbitrary real numbers; they must lie on a grid. The grid spacing is the **tick size**, written θ . If $\theta = 0.1$, then 100.2 is a legal price and 100.25 is not. Similarly, sizes must be multiples of a **lot size**. Tick size matters enormously for competition between market makers, for a reason we will see in Section 6.2.4.

2.5 Fees

Exchanges charge a **taker fee** (a small percentage of the trade value) to whoever removed liquidity, and often pay a **maker rebate** (a small negative fee) to whoever provided it, to encourage people to post resting orders. Fees are quoted in **basis points (bps)**: $1 \text{ bp} = 0.01\% = 0.0001$. A 2.5 bp taker fee on a \$10,000 trade is \$2.50.

Notation decoded

m_t	mid-price at time t
θ (or θ_t)	tick size (may change over time, see Section 6.2.4)
$\mathcal{B}_t$	the entire order book at time t (all resting orders)
r	maker rebate per unit traded
f	taker fee per unit traded

3 Part II: Market Makers and Why They Get Hurt

3.1 What a market maker does

A **market maker** (MM) is a trader who *simultaneously* posts a bid and an ask, all the time. They are not betting on the direction of the price; they are selling immediacy to everyone else and collecting the spread.

Example: the ideal day for a market maker

The true value of a coin is exactly 100 all day. An MM posts a bid at 99.9 and an ask at 100.1 (spread 0.2). A taker arrives and sells to the MM at 99.9. Then another taker arrives and buys from the MM at 100.1. The MM bought at 99.9 and sold at 100.1: profit 0.2. Repeat a thousand times per day. This is the business.

3.2 Risk 1: inventory

In practice buys and sells don't alternate neatly. If ten takers in a row sell to the MM, the MM now owns ten units. That is **inventory**, written I . Inventory is risky: if the price drops 1%, the MM loses 1% of the value of everything it holds, which can wipe out days of spread income. Market makers therefore *skew* their quotes when they hold inventory: if they own too much, they lower both their bid and ask to make selling more likely and buying less likely.

In our model we don't try to capture skewing in full detail. Instead we add a penalty ϕI^2 to the market maker's objective: holding inventory I costs ϕI^2 per period. Because it is quadratic, small inventories are nearly free and large ones are very expensive, which pushes the learning agent to keep I near zero. There is also a hard cap $|I| \leq \bar{I}$ (the MM literally cannot hold more than $\bar{I}$ units).

3.3 Risk 2: adverse selection — the central idea

Now the important one. Some of the people who trade with the MM *know something*. Perhaps they have seen that the price on another exchange just jumped. When such an **informed trader** buys from the MM, it is precisely because the price is about to go up. The MM sold at 100.1 and then watches the price go to 101. The MM “won” the spread but lost 0.9 on the position. This is **adverse selection**: the counterparty selection is adverse to you because the people most eager to trade are exactly those who know you are mispriced.

Intuition

Imagine you run a used-car lot and offer to buy any car for \$5,000 and sell any car for \$5,200. Ordinary people sell you cars worth roughly \$5,100 on average; fine. But mechanics who know their car is a lemon worth \$3,000 will *rush* to sell it to you at \$5,000. The people who most want to trade with you are the ones you least want to trade with. To survive you must lower your buy price and raise your sell price—widen your spread—to cover the losses to the mechanics.

The fraction of trades that come from informed traders is called **toxicity**. In our proposal this is the parameter α : each arriving taker is informed with probability α and uninformed with probability $1 - \alpha$. A market with $\alpha = 0.5$ is very toxic (half the flow knows something); $\alpha = 0.05$ is benign.

3.4 The Glosten–Milgrom model: computing the fair spread

Glosten and Milgrom (1985) wrote down the simplest possible model of this situation, and it is the backbone of our simulator. Here it is with numbers.

Example: Glosten–Milgrom with numbers

Setup. The asset’s true value V is either $V_L = 90$ or $V_H = 110$, each with probability $1/2$, so $\mathbb{E}[V] = 100$. A single taker arrives. With probability $\alpha = 0.3$ the taker is informed: they know V and buy if $V = 110$, sell if $V = 90$. With probability 0.7 the taker is uninformed and flips a coin: buys or sells with probability $1/2$ each, regardless of V .

Question. A competitive market maker wants to set an ask price a such that it makes *zero* expected profit on a sale. What is a ?

Answer. The MM should set $a = \mathbb{E}[V \mid \text{taker buys}]$: the expected value of the asset *given that* someone chose to buy. Use Bayes’ rule.

$$\begin{aligned}\mathbb{P}(\text{buy} \mid V = 110) &= \alpha \cdot 1 + (1 - \alpha) \cdot \frac{1}{2} = 0.3 + 0.35 = 0.65, \\ \mathbb{P}(\text{buy} \mid V = 90) &= \alpha \cdot 0 + (1 - \alpha) \cdot \frac{1}{2} = 0.35.\end{aligned}$$

So

$$\mathbb{P}(V = 110 \mid \text{buy}) = \frac{0.5 \times 0.65}{0.5 \times 0.65 + 0.5 \times 0.35} = \frac{0.65}{1.00} = 0.65,$$

and therefore

$$a = \mathbb{E}[V \mid \text{buy}] = 0.65 \times 110 + 0.35 \times 90 = 71.5 + 31.5 = 103.$$

By symmetry the zero-profit bid is $b = \mathbb{E}[V \mid \text{sell}] = 97$. The **competitive spread** is $a - b = 6$.

Check the logic. Why 103 and not 100? Because a buy order is evidence the value is high. If the MM sold at 100, then in the 0.65 fraction of buys where $V = 110$ it loses 10, and in the 0.35 fraction where $V = 90$ it gains 10; net $0.65(-10) + 0.35(10) = -3$ per sale. Selling at 103 makes that net exactly zero.

Example: how the spread depends on toxicity

Redo the calculation for different α :

α	$\mathbb{P}(V_H \mid \text{buy})$	competitive ask a	competitive spread $a - b$
0.0	0.50	100	0
0.1	0.55	101	2
0.3	0.65	103	6
0.5	0.75	105	10
1.0	1.00	110	20

The competitive spread is *exactly* 20α in this example. More informed flow means wider spreads, even with perfect competition and zero profit. **This is the crux of the whole project:** a wide spread does not by itself mean the market makers are greedy. It might mean the flow is toxic.

3.5 Belief updating after each trade

In the example above the MM updated its belief about V after one trade. In our simulator V is drawn once per *episode* and stays fixed for $T = 100$ periods, so after every trade the whole market updates its belief. We write the public belief as $\mu_t = \mathbb{E}[V \mid \text{everything printed so far}]$. It starts at $\mu_1 = 100$ and moves toward the truth as informed trades accumulate.

Example: two buys in a row

Start with $\mathbb{P}(V_H) = 0.5$, $\alpha = 0.3$. After one buy, $\mathbb{P}(V_H) = 0.65$ (computed above). Now a second buy arrives. Bayes' rule again, using 0.65 as the new prior:

$$\mathbb{P}(V_H \mid \text{buy, buy}) = \frac{0.65 \times 0.65}{0.65 \times 0.65 + 0.35 \times 0.35} = \frac{0.4225}{0.4225 + 0.1225} = 0.775.$$

So $\mu = 0.775 \times 110 + 0.225 \times 90 = 105.5$. The mid-price has drifted up from 100 to 103 to 105.5: the market is “discovering” that $V = 110$. The quantity $|\mu_t - V|$ measures how far the market is from the truth, and how fast it shrinks is **price discovery**. When we say collusion “slows price discovery,” we mean $|\mu_t - V|$ shrinks more slowly.

3.6 Markouts: measuring adverse selection from data

How would you measure toxicity from real trading records, where you never see V ? Use the **mark-out**: after a market maker's fill at price p , look at the mid-price Δ seconds (or blocks) later, $m_{t+\Delta}$. If the MM sold at p and the mid then rose above p , the MM was adversely selected.

Formally, let $s = +1$ if the taker bought (MM sold) and $s = -1$ if the taker sold (MM bought). Define

$$\begin{aligned} \text{effective spread} &= 2s(p - m_t) && \text{(how far from mid the taker paid)} \\ \text{realized spread} &= 2s(p - m_{t+\Delta}) && \text{(what the MM actually kept)} \\ \text{price impact} &= 2s(m_{t+\Delta} - m_t) && \text{(how far the price moved against the MM)} \end{aligned}$$

and these satisfy $\text{effective} = \text{realized} + \text{impact}$. (The factor of 2 is a convention: it converts a half-spread into a full spread.)

Example: one fill, two futures

The mid is $m_t = 100.0$ and the MM's ask is 100.1. A taker buys, so $s = +1$, $p = 100.1$. The effective spread is $2(100.1 - 100) = 0.2$.

Future A (benign taker): ten seconds later the mid is still 100.0. Realized spread = $2(100.1 - 100.0) = 0.2$; price impact = 0. The MM kept the whole spread.

Future B (informed taker): ten seconds later the mid is 100.8. Realized spread = $2(100.1 - 100.8) = -1.4$; price impact = $2(100.8 - 100.0) = 1.6$. The MM “earned” 0.2 on paper and lost 1.6 on the move. Net -1.4 .

Averaging the realized spread over thousands of fills tells you how much the MM keeps *after* adverse selection. Averaging the price impact tells you how toxic the flow is.

Intuition

Under perfect competition the expected realized spread is roughly **zero**: the whole quoted spread is eaten by adverse selection (that is what “zero expected profit” means in Glosten–Milgrom). A persistently *positive* realized spread is **rent**: money the MM keeps beyond what covering its adverse selection requires. This is how a trader, using only public prints, can start to tell rent from toxicity. That is Research Question 1.

3.7 The rent-or-toxicity puzzle, stated plainly

Put the last two subsections together. You are a trader looking at two markets, both with quoted spread 6:

- Market A: $\alpha = 0.3$. Spread 6 is exactly competitive. If you enter and quote spread 4, you will win all the trades and lose money on every informed one; expected profit per trade is negative.
- Market B: $\alpha = 0.05$. The competitive spread is 1. Spread 6 means the incumbents are keeping 5 of rent per round trip. If you enter and quote 5, you win the trades and keep 4 of rent.

Same observable, opposite correct action. Only the *markouts* (or a model of the incumbents) distinguish them. Colliard, Foucault and Lovo report a surprising twist: their learning market makers earned the *most* rent when adverse selection was *lowest*. If that holds on real venues, a trader who enters “where spreads look fat” is walking into Market A, and the rent is sitting in the markets that look boring.

4 Part III: Games, Equilibrium, and Collusion

4.1 What “strategic” means

So far one MM faced random takers. Now put N market makers in the same book. Each one's profit depends on what the others quote: if you post an ask at 103 and I post 102.9, I get the sale and you get nothing. This interdependence is what makes it a **game**. A **strategy** is a complete rule for what to quote in every situation. A **Nash equilibrium** is a set of strategies, one per player, such that nobody can increase their profit by changing only their own strategy.

4.2 Bertrand competition: why the competitive spread is the equilibrium

Suppose two MMs choose asks on a tick grid, and the taker always buys from the cheaper one (ties split). Suppose the zero-profit ask is 103 and $\theta = 0.1$.

- If both quote 105, each earns 2 on half the trades. But if I quote 104.9 I win *all* the trades at 1.9 each, which is better than $2 \times \frac{1}{2} = 1$. So 105 is not an equilibrium.
- The same undercutting argument works at any ask above 103.1. It stops when undercutting no longer pays: at an ask of 103.1 (one tick of profit), undercutting to 103.0 yields zero.

So the unique equilibrium is (roughly) the competitive ask, with at most one tick of profit. This “race to the bottom” is **Bertrand competition**, and it is why economists say competitive market makers earn no rent. The **joint-monopoly** outcome—what a single firm owning both MMs would choose—is the profit-maximizing spread, much wider.

4.3 Repeated games and collusion

The Bertrand argument assumed a one-shot game. But market makers face each other thousands of times a day. In a **repeated game**, new equilibria appear because players can *punish*. Consider the **grim-trigger strategy**: “Quote the monopoly spread as long as everyone else does. The moment anyone undercuts, quote the competitive spread forever.” If both players use this rule:

- Undercutting today wins one big payday (all the trades at nearly the monopoly spread).
- But it triggers zero profit forever after.

If players value the future enough (the **discount factor** γ is close to 1), the one-time gain is smaller than the lost future, and nobody undercuts. Wide spreads are sustained with *no communication at all*. Economists call this **tacit collusion**. Legally, collusion is defined by exactly this reward–punishment structure, not by whether prices are high.

Example: is patience enough?

Monopoly spread gives each of two MMs 2 per period. Undercutting gives 3.9 once (all trades at spread 3.9) and then 0 forever. Cooperating forever is worth $2 + 2\gamma + 2\gamma^2 + \dots = \frac{2}{1-\gamma}$. Deviating is worth 3.9. Cooperation survives if $\frac{2}{1-\gamma} > 3.9$, i.e. $\gamma > 0.49$. With $\gamma = 0.95$ (typical in RL) the collusive outcome is easily sustainable. This is why the learning agents in the papers we cite so often end up there.

4.4 Price-trigger strategies and the impulse-response test

A “price-trigger” strategy is the market version of grim trigger: an MM watches competitors’ *quotes*, and a competitor’s undercut triggers a punishment phase (a price war). How do you verify from the outside that agents are *really* colluding, rather than just happening to quote wide? Calvano et al. (2020) introduced the **impulse-response test**: freeze the simulation, *force* one agent to undercut for one period, then release everyone and watch. If the others respond by slashing their spreads for several periods (punishment) and then spreads gradually return to the wide level, you have observed the reward–punishment mechanism directly. If nothing happens, the wide spreads were not collusion in the legal sense—perhaps just under-exploration (Section 5.3).

4.5 The collusion index Δ

To put “how collusive” on a 0-to-1 scale:

$$\Delta = \frac{\bar{\pi} - \pi^C}{\pi^M - \pi^C},$$

where $\bar{\pi}$ is the average profit the agents actually earn, π^C is the profit in the competitive (Nash/Bertrand) outcome, and π^M is the joint-monopoly profit. $\Delta = 0$ means fully competitive, $\Delta = 1$ means fully collusive.

Example

Competitive profit is 0.5 per period (one tick), monopoly profit is 4.5, and the learners average 2.5. Then $\Delta = (2.5 - 0.5)/(4.5 - 0.5) = 0.5$: halfway to a cartel. Because our simulator is built on Glosten–Milgrom, we can compute π^C and π^M *exactly*, which is why the proposal insists on the stylized model (see Feedback question 1).

5 Part IV: Learning Agents

5.1 Reinforcement learning in one paragraph

An agent lives in an environment. At each step it sees a **state** s , picks an **action** a , receives a **reward** R , and moves to a new state. It wants to maximize the discounted sum of future rewards $\sum_t \gamma^t R_t$. It does not know the rules of the environment; it learns by trying things. In our setting the state is (a summary of) the order book and the agent’s inventory, the action is the pair of quotes, and the reward is trading profit minus the inventory penalty. That is exactly the objective in the proposal:

$$\max \mathbb{E} \sum_t \gamma^t [\pi_t^i - \phi(I_t^i)^2].$$

5.2 Q-learning, with a worked example

Q-learning keeps a table $Q(s, a)$: the agent’s current estimate of “how much total future reward I get if I take action a in state s and behave sensibly afterwards.” After each step it nudges the estimate toward what it just observed:

$$Q(s, a) \leftarrow (1 - \eta) Q(s, a) + \eta \left[R + \gamma \max_{a'} Q(s', a') \right],$$

where η is the **learning rate** (how big a nudge) and s' is the next state. To act, the agent usually picks the action with the highest Q , but with probability ε it picks a random action instead. This is **ε -greedy exploration**; without it the agent would never discover better actions.

Example: three steps of Q-learning for a market maker

Simplify: the state is just “competitor’s last spread” $\in \{\text{narrow}, \text{wide}\}$, and the action is my spread $\in \{2, 4, 6\}$. All Q values start at 0. Set $\eta = 0.5$, $\gamma = 0.9$.

Step 1. State: wide. I pick spread 6 (random exploration). I earn $R = 3$ (half the trades at spread 6). Next state: wide. Update: $Q(\text{wide}, 6) \leftarrow 0.5 \cdot 0 + 0.5[3 + 0.9 \cdot 0] = 1.5$.

Step 2. State: wide. Greedy choice is now 6 ($Q = 1.5$ beats 0). I earn $R = 3$ again. Update: $Q(\text{wide}, 6) \leftarrow 0.5 \cdot 1.5 + 0.5[3 + 0.9 \cdot 1.5] = 0.75 + 0.5 \cdot 4.35 = 2.925$.

Step 3. Exploration fires; I try spread 4. I win all trades: $R = 4$. But the competitor responds next period by going narrow (a punishment). Next state: narrow, where all my Q values are still 0. Update: $Q(\text{wide}, 4) \leftarrow 0.5 \cdot 0 + 0.5[4 + 0.9 \cdot 0] = 2.0$.

After three steps, $Q(\text{wide}, 6) = 2.925 > Q(\text{wide}, 4) = 2.0$, so the greedy choice is to keep quoting wide when the competitor quotes wide, even though undercutting paid more *this period*. As the agent learns that “narrow” states lead to low rewards, that gap widens. This is the seed of learned collusion.

5.3 Why learning agents end up colluding (two different mechanisms)

The literature has found two distinct reasons, and our impulse-response test is designed to tell them apart.

1. **Genuine reward–punishment.** As in the example: the agents learn that undercutting is followed by price wars, so they don’t. The impulse-response test shows punishment.
2. **Under-exploration / noisy feedback.** Colliard, Foucault and Lovo argue their agents never learned to undercut simply because the payoff from undercutting is noisy (in a toxic market you sometimes lose even when you undercut correctly) and exploration is limited, so the agent never accumulated enough evidence that undercutting works. Nobody is punishing anybody; the agents are just under-informed. The impulse-response test shows *no* punishment.

For a trader the difference matters: mechanism 1 means an entrant *will* be punished; mechanism 2 means the incumbents are simply soft.

5.4 No-regret (Hedge) learners

A second family of learners doesn’t keep a Q -table. A **no-regret** algorithm keeps a weight on each possible action and, after every round, raises the weights of actions that *would have* done well and lowers the others. **Hedge** does this with exponential weights: $w_a \leftarrow w_a \exp(\eta \text{payoff}_a)$, then plays each action with probability proportional to w_a . “No regret” means: in the long run, the algorithm does at least as well as the best *fixed* action would have done in hindsight.

Example

Three spreads $\{2, 4, 6\}$, all weights start at 1. Round 1 payoffs (had I played each): $\{1.5, 2.5, 1.0\}$. With $\eta = 1$, weights become $\{e^{1.5}, e^{2.5}, e^{1.0}\} \approx \{4.5, 12.2, 2.7\}$; probabilities $\approx \{0.23, 0.63, 0.14\}$. The algorithm shifts toward spread 4 but keeps trying the others.

The important theoretical fact (Deng, Schneider and Sivan, 2019) is that a no-regret learner is *predictable in a specific way*, and a clever opponent who knows it is facing one can **steer** it: by choosing its own actions to shape the learner’s observed payoffs, the opponent can make the learner converge to whatever the opponent finds most profitable. This is our “steering attacker.”

5.5 Deep RL (PPO) in one line

When the state is too rich for a table, the Q -values or the policy are represented by a neural network. **PPO** (proximal policy optimization) is the standard, stable algorithm for training such a policy. We include it as an incumbent type to check that our conclusions don't depend on the toy tabular learner. Cont and Xiong's collusion result used a deep multi-agent method of this kind.

5.6 Best response, exploitability, and PSRO

Freeze the incumbents' learned policies. Now ask: *what is the most profitable thing a new agent could do against exactly these policies?* That optimal counter-strategy is the **best response**, and the profit it earns is the incumbents' **exploitability**. In our proposal we report it in **bps per unit volume**: the number of basis points of value the best responder extracts per dollar traded.

Example

Incumbents' learned policies leak, on average, 0.30 price units per 100 traded to a best-responding entrant. That is $0.30/100 = 0.003 = 30$ bps. If a market trades \$1B a day, that is \$300,000 a day on the table.

Computing an exact best response is usually impossible, so we approximate it by training an attacker (with RL) against frozen incumbents. **PSRO** (policy-space response oracles) repeats this: train a best response, add it to the pool of opponents, retrain, and so on. The later generations are attackers the incumbents *never saw*, which is what the rubric means by "previously unseen attack strategies."

6 Part V: Perpetual Futures and the Hyperliquid Exchange

6.1 What is a perpetual future?

A **perpetual future** ("perp") is a contract whose price tracks an underlying asset (say Bitcoin) but which you can hold forever and trade with **leverage**: you put down, say, \$1,000 of **margin** and control \$10,000 of exposure ($10\times$ leverage). To keep the perp's price near the real asset, exchanges use a **funding rate**: periodically, whichever side is more crowded pays the other. For our purposes the key facts are:

- The exchange computes a **mark price** (a blend of an external index and the local book). This plays the role of V , the fundamental.
- Because of leverage, a small adverse move can exhaust your margin, at which point the exchange forcibly closes your position: a **liquidation**. Liquidations are *forced market orders*, which is why they matter for market makers.

6.2 Hyperliquid in plain words

Hyperliquid is an exchange for perps that runs its order book *on a blockchain*: every order, every cancel, every fill is a transaction that everyone can see, tagged with the wallet address of the sender. Blocks are produced continuously at sub-second intervals, and inside each block orders are matched

deterministically by price–time priority. This gives us six mechanics that are new relative to every model in the literature.

6.2.1 Mechanic 1: persistent wallet identity

On stock exchanges, orders are anonymous: you see a bid for 500 shares but not who posted it. On Hyperliquid you see the *wallet address*: a persistent pseudonym, not a legal identity (one firm can run several wallets, so say “wallet identity,” never “trader identity”). This matters because **targeted punishment requires knowing who defected**. With two colluders, anonymity doesn’t matter (if I didn’t undercut, it must have been you). With three or more, an anonymous book makes it hard to know whom to punish, which is why economists expect collusion to be harder. Cartea, Chang and Graumans found that on an anonymous equity venue, high-frequency market makers *pay* for identity: they post unusually large orders as a recognizable signature so that other MMs can avoid trading against them. On Hyperliquid identity is free, so we can check directly whether MMs avoid each other.

Example: the wallet-pair test

Take all trades in December. Build a matrix whose (i, j) entry is how many times wallet i traded against wallet j . If MMs were matching at random, the entry for two MMs would be proportional to their volumes. If it is far *below* that, they are avoiding each other—consistent with the collusive equilibrium in the paper above.

Common confusion

“Public” means public *after* the block commits. A market maker cannot see who is about to hit its quote and refuse the trade. What it can do is use each wallet’s *past* fills (public after commitment) to predict how toxic that wallet’s future orders are likely to be, and shade its quotes accordingly. Zhai (2026) shows on 17 billion Hyperliquid messages that this works: a wallet’s toxicity is persistent from one ten-day window to the next (rank correlation 0.52), and adding wallet-history features improves short-horizon return forecasts out of sample. Zhai stops there and says explicitly that turning the forecast into trading profit would need a dynamic market-making model with latency, queue position, fills, fees and inventory—which is what our simulator is.

6.2.2 Mechanic 2: request budgets (rate limits)

To stop spam, Hyperliquid limits how many *actions* (order placements, cancels) an address can send. The rule is unusual: an address earns **one action per USDC of cumulative traded volume**, starting from a buffer of 10,000. When the budget is exhausted, the address may send only one action every 10 seconds (cancels get extra headroom so you can always exit). There is also a cap on open orders.

Example: a budget in action

A new market maker starts with 10,000 actions. Each block it updates both its bid and its ask: 2 cancels + 2 new orders = 4 actions. If a block is 0.1 s, that is 40 actions per second

and the buffer lasts 250 seconds—about four minutes. To keep quoting at that rate it must *fill* 40 USDC of volume per second. A market maker that quotes aggressively but doesn’t get filled runs out of budget and freezes. In the proposal this is the state variable B_t^i , and the constraint that quoting must be “earned by filling” is unlike anything in prior models.

Example: induced self-exhaustion

Nobody can spend another wallet’s budget; the limit is per user. But a reactive policy spends its *own* budget whenever it chooses to respond. Suppose MM A follows a rule “always match the best quote.” An adversary Z posts a bid one tick better; A cancels-and-matches (2 of A ’s actions). Z cancels; A cancels-and-reverts (2 more). Z spent 2 actions, A spent 4. Repeat, and A ’s own policy drains A ’s budget until A is throttled to one request per 10 seconds and its quotes go stale. The attack is a *stimulus*; the damage is done by the victim’s reaction rule. That makes it a real attack-and-defence game: the defence is a policy that does not react to every flicker (hysteresis, a minimum quote lifetime, a reserve budget). Identified-trader data show flickering quotes do elicit responses from other participants (Brogaard, Cartea, Chang and Graumans 2026), so the stimulus cannot be assumed harmless either. Whether to include it is Feedback question 2.

6.2.3 Mechanic 3: priced latency

In traditional markets, being faster than competitors requires expensive hardware, and the race to be fastest (the “arms race,” Budish et al.) has no explicit price. On Hyperliquid, two rules make speed a purchasable good:

- within a block, **all cancels are processed before any order that would execute immediately**, so a market maker who sees danger and cancels always beats a sniper who tries to hit its stale quote in the same block;
- a trader can attach a **priority fee** to an order. For an *aggressive* order (one that executes immediately) each basis point of fee cuts end-to-end latency by roughly 45 ms, up to 8 bp. For a *resting* order the fee works differently: it buys a better position in the queue within a short window. So for a market maker, whose orders rest, priority is a way to buy queue position, not raw speed.

This turns the sniping game into an *auction* with a known price schedule, which we can put in the simulator and calibrate.

Example

A sniper sees the index price jump and wants to hit an MM’s stale ask. The MM sees the same jump and sends a cancel. If both land in the same block, the cancel wins. The sniper’s only recourse is to land in an *earlier* block, which means paying priority fees. If the stale quote is mispriced by 3 bps and the sniper needs 2 bps of priority fee to win the race, the sniper nets 1 bp; if it needs 4 bps, sniping is unprofitable and the MM is safe. Sniping profitability is now a simple inequality in observable quantities.

6.2.4 Mechanic 4: tick-size discontinuities

Hyperliquid prices may have at most **five significant figures**. So:

- at a price of 9,999, the finest legal step is 0.1 (five significant figures: 9999.9);
- at 10,000, the finest legal step is 1 (five significant figures: 10000; 10000.1 has six).

So the effective tick size θ changes by a factor of 10 when the price crosses a power of ten. The full venue rule also caps the number of decimal places per asset, and integer prices are always legal (which is why BTC above \$100,000 has an effective tick of \$1), so the simulator computes the legal price grid from the rule for each asset and price range rather than assuming a clean $10\times$ jump everywhere.

Why does tick size matter for collusion? In the Bertrand argument, undercutting works because I can shave a little off your price. If the tick is large relative to the competitive spread, “a little” is not available: with a tick of 1 and a competitive spread of 0.6, the narrowest legal spread is 1 and every MM sits at it, splitting the flow, with no way to compete. Cartea, Chang and Penalva showed that learning agents collude more at larger ticks. Hyperliquid hands us dozens of coins crossing powers of ten at random times: each crossing is a natural experiment where the tick changes and nothing else does.

6.2.5 Mechanic 5: HLP, an inspiration for the fixed-policy benchmark

The protocol itself runs a market-making vault called **HLP**. Anyone can deposit into it; it quotes across all markets, absorbs liquidations that the book cannot, and its positions are visible on-chain. Its strategies are proprietary and undisclosed; the documentation says only that it runs “multiple market-making strategies.” We therefore do *not* claim to know HLP’s policy. The course scope requires a fixed-policy agent for comparison; ours is a fixed-spread, inventory-skewing quoter *calibrated to what HLP observably does* (its positions and fills are on-chain), and “how much can adaptive agents extract from an HLP-like quoter?” is the question.

6.2.6 Mechanic 6: liquidation cascades and 10 October 2025

When a leveraged trader’s margin runs out, Hyperliquid sends a forced market order to the book; if the book cannot absorb it, the position is transferred to the backstop vault. On 10 October 2025 a macro announcement triggered the largest liquidation event in crypto history: over \$19B of positions were closed across venues, with CoinGlass attributing about \$10.3B to Hyperliquid. Separate multi-venue microstructure data (Amberdata) report spreads roughly $30\times$ normal and depth down about 98% during the worst 40 minutes; those two numbers come from different sources and the spread figure is not Hyperliquid-specific, which is why we will compute Hyperliquid’s own spread and depth path from the order-level data. This is precisely the “rare or high-impact scenario” the course asks for, and because it is on-chain, we can see exactly what every market maker did. In the model, liquidations are the *nonstrategic disruption*: they arrive at a rate $\lambda(\sigma_t)$ that rises with volatility, with sizes drawn from a distribution F calibrated to the October data.

6.2.7 The dataset

An Oxford group ran a Hyperliquid node and published what they call Level-4 data: for BTC, ETH and SOL in December 2025, every order’s full lifecycle (placed, modified, cancelled, rejected, filled)

with wallet IDs, counterparties on both sides of every trade, and each wallet’s inventory after each trade; plus all trades on 250+ contracts from October 2025 through January 2026, spanning the cascade. Conventional academic datasets (e.g. LOBSTER for NASDAQ) have none of the identity or rejection information. This is the empirical backbone of the project.

7 Part VI: The Model, Line by Line

What the deliverable says

“One perpetual contract on a Hyperliquid-style CLOB. Time is discrete in blocks $t = 1, \dots, T$. An episode is $T = 100$ blocks; learning runs across 10^4 – 10^5 episodes.”

A **block** is one tick of our clock: everything that happens “at time t ” is processed together, then t becomes $t + 1$. Hyperliquid does process orders in discrete blocks, but the blocks arrive on an irregular clock (median gap about 68 ms) and order priority within the block logic is continuous in the priority fee, so block time is a tractable event-time *approximation*, not an exact model. An **episode** is one “game” of 100 blocks with a fixed true value V ; at the end V is revealed and profits settle. The agents then play tens of thousands of episodes so that their learning converges.

7.1 The agents

What the deliverable says

“Incumbent MMs $i \in \{1, \dots, N\}$. Action $a_t^i = (\delta_t^{b,i}, \delta_t^{a,i}, q_t^i)$: bid/ask offsets from m_t on the tick grid, and displayed size.”

Each incumbent chooses three numbers per block: how far below the mid to bid (δ^b), how far above the mid to ask (δ^a), and how many units to show (q). If $m_t = 100$, $\delta^b = 0.3$, $\delta^a = 0.2$, $q = 5$, the MM posts “buy 5 at 99.7, sell 5 at 100.2.” Offsets must be multiples of the tick. Choosing offsets rather than absolute prices makes the action space small and the learning tractable.

What the deliverable says

“Strategic entrant (the trader). One MM who observes the public tape and best-responds: infers whether the spread is rent or toxicity and chooses offsets and sizes.”

The **tape** is the public record of prints (trades) and quotes. The entrant is the protagonist. It is not a learner in the incumbents’ sense; it is whatever policy *we* design, from a naive rule to a full opponent model. Section 3.7 is its problem statement.

What the deliverable says

“Takers. One arrival per block: informed with probability α (trades in the direction of $V - m_t$), else a liquidity trader with private valuation $m_t + L$, $L \sim G$.”

This is Glosten–Milgrom from Section 3.4, with one refinement: the uninformed trader is not a pure coin flip but has a personal reason to trade, modelled as a random valuation $m_t + L$. They

buy only if the best ask is below their valuation, and sell only if the best bid is above it. G is the distribution of L ; a wide G means many traders willing to pay large spreads (lucrative), a narrow G means they walk away if spreads are wide. The remark “on Hyperliquid this is empirically latency arbitrage against Binance/OKX mid-prices” tells you where informed flow actually comes from in crypto: traders who see the price move on a bigger exchange a few milliseconds earlier.

What the deliverable says

“Predatory taker. A strategic informed trader who chooses timing and size to exploit incumbents’ learning dynamics.”

The ordinary informed taker trades the moment it knows something. The predatory one is patient. It knows the incumbents are learners and asks: when are they *most* mispriced? Two examples from the proposal: (a) provoke a price war (by briefly acting as an undercutting MM), then trade heavily while spreads are artificially tight; (b) wait until a competitor has exhausted its request budget (Section 6.2.2) and trade against its frozen quotes.

What the deliverable says

“Nonstrategic disruption: liquidation flow. With hazard $\lambda(\sigma_t)$ a forced market order of size $S \sim F$ arrives.”

A **hazard** is a per-block probability of an event. σ_t is a volatility state; when it is high, liquidations are more likely. Each liquidation is a market order of random size S . It is “nonstrategic” because nobody chooses it: it is the environment throwing a punch. The rubric requires such a baseline so we can separate “damage from randomness” from “damage from adversaries.”

7.2 State, payoffs, sequence

Notation decoded

s_t	the state: $(\mathcal{B}_t, m_t, \mu_t, \{I_t^i, B_t^i\}_i, \sigma_t)$
$\mathcal{B}_t$	the visible order book (with wallet IDs in the transparent regime)
$\mu_t = \mathbb{E}[V \mid \text{tape}]$	the market’s current best guess of the true value
I_t^i	inventory of MM i
B_t^i	remaining request budget of MM i
σ_t	volatility state (drives the liquidation hazard)
π_t^i	trading profit of MM i in block t
γ	discount factor (how much the agent values next block vs this one)
ϕ	inventory-penalty weight
$\bar{I}$	hard inventory cap

What the deliverable says

“Payoffs. Per fill $\pi_t^i = (p - V)$ when selling, $(V - p)$ when buying, plus maker rebate r minus taker fee f for aggressive orders.”

When you sell one unit at price p and its true value is V , you gained $p - V$ (positive if you sold above value). When you buy at p , you gained $V - p$. The rebate and fee are added per unit. Using V rather than the later mid-price makes the accounting exact inside the simulator; on real data we substitute the markup (Section 3.6) because V is unobservable.

What the deliverable says

“Sequence. MMs quote simultaneously $\rightarrow$ taker/liquidation arrives $\rightarrow$ match $\rightarrow$ print, belief update, budgets decrement $\rightarrow$ learners update $\rightarrow$ episode end: V revealed, PnL settled.”

Example: three blocks of one episode, end to end

Parameters: $V \in \{90, 110\}$, $\alpha = 0.3$, $\theta = 0.5$, two Q-learning incumbents, one entrant, $N = 3$ quoters total. This episode’s draw: $V = 110$ (unknown to everyone). $\mu_1 = 100$.

Block 1. Quotes: MM1 bid 97 / ask 103; MM2 bid 97 / ask 103; entrant bid 97.5 / ask 102.5. Best bid 97.5 (entrant), best ask 102.5 (entrant). Taker draw: informed (prob. 0.3)—yes. Informed with $V = 110$ buys at the best ask 102.5. Entrant sells one unit: $\pi = 102.5 - 110 = -7.5$, inventory $I = -1$. Budgets: each MM spent 2 actions posting. Print: buy at 102.5. Belief update: $\mu_2 = 103$ (one buy observed, as in Section 3.4).

Block 2. Mid is now 103. Quotes re-centred: MM1 100/106; MM2 100/106; entrant, holding short inventory, skews up: 101/105. Taker draw: uninformed, $L = -1.5$, valuation 101.5. Best bid is 101 (entrant) < 101.5 , so the uninformed trader is willing to sell at 101. Entrant buys: $\pi = 110 - 101 = +9$, inventory back to 0. Print: sell at 101. Belief update: a sell is weak evidence for $V = 90$; $\mu_3 \approx 101$.

Block 3. Liquidation hazard fires: forced sell of $S = 6$ units. Best bids: entrant 99 ($q = 2$), MM1 98 ($q = 5$). Fills: 2 at 99 to the entrant, 4 at 98 to MM1. MM1: $\pi = 4 \times (110 - 98) = +48$ (a windfall this time, but V could have been 90), $I = 4$, and now pays penalty $\phi \cdot 16$ each block until it unwinds.

Learners update. Each incumbent nudges its Q -table with the reward it got. **Episode continues** to block 100, then $V = 110$ is revealed and all inventory is valued at 110.

7.3 Benchmarks and metrics

What the deliverable says

“Competitive Glosten–Milgrom quotes on the grid (ask = $\lceil \mathbb{E}[V \mid \text{buy}, s_t] \rceil_\theta$); joint-monopoly quotes; collusion index Δ .”

$\lceil x \rceil_\theta$ means “round x up to the next multiple of the tick θ .” A competitive MM computes the zero-profit ask (Section 3.4) and rounds up to the grid; on a coarse grid that rounding is itself a small rent, which is why tick size matters. The joint-monopoly quotes are found by brute-force search over the grid for the spread that maximizes total MM profit. Then Δ is computed as in Part III.

The metrics listed in the proposal, in plain words:

- **quoted spread:** best ask minus best bid;
- **effective / realized spread, price impact:** Section 3.6, at horizons of 1, 10 and 60 blocks;

- **taker welfare**: total surplus of uninformed traders (their valuation minus what they paid); wider spreads lower it;
- **price-discovery error** $|\mu_t - V|$: how wrong the market is;
- **MM PnL mean / variance / tail**: average profit, its spread, and the worst 5% of episodes;
- **exploitability**: Section 5.6.

7.4 The simplifying assumptions, and why each is acceptable for now

- *One asset*. Real MMs quote many contracts at once. Extension: K correlated assets (Feedback question 3).
- *Unit taker size*. Every taker trades one unit. Keeps the book simple; liquidations are the exception.
- *Risk-neutral MMs*. They care about expected profit, not variance; the inventory penalty is a stand-in for risk aversion.
- *Exogenous taker mix*. α is a parameter, not something that responds to spreads. In reality wide spreads deter uninformed traders; the $L \sim G$ mechanism captures part of this.
- *No cross-venue hedging*. Real MMs offset inventory on Binance. Ignored for now.
- *Calibrated, not endogenous, liquidations*. We draw liquidation sizes from data rather than modelling the leverage that produces them.

8 Part VII: The Research Questions and the Attacks

8.1 RQ1: rent or toxicity from the tape

The question. Can an entrant, seeing only public prints, tell Market A from Market B in Section 3.7, and how much money is that worth?

The three entrant policies compared.

1. *Competitive-GM quoting (benchmark)*. Assume the incumbents are competitive, compute the zero-profit spread from an estimate of α , quote that. This is what a textbook MM does. It makes no attempt to detect rent.
2. *Markout-based inference*. Track the incumbents' realized spread. If it is persistently positive (rent), quote one tick inside them; if it is near zero (toxicity), quote at or wider than competitive. This uses only public data.
3. *Learned opponent model*. Fit a model of how each incumbent responds to the state (including how it reacts to being undercut) and best-respond to that model. This can anticipate punishment.

Measured. Entrant profit, share of trades won, and *rounds-to-identification*: how many blocks until the entrant's estimate of "rent vs toxicity" is correct with high confidence. Run at $\alpha \in \{0.1, 0.3, 0.5\}$.

The prediction to test. If Colliard et al. are right that rent is largest when α is smallest, then a rule of thumb like "enter where quoted spreads are wide" fails, and the markout rule should beat it by a wide margin.

8.2 RQ2: exploitability of learned quoters

The question. Freeze each type of incumbent. How much can a best-responding adversary take from it? And does a colluding group of incumbents leak *more* than a competitive one?

The incumbent types. Tabular Q-learning; Hedge (no-regret); PPO (deep RL); an HLP-inspired fixed rule.

The attackers, each explained.

- *One-tick undercutter.* Always quote one tick inside the best incumbent quote. The simplest possible attack. Works if incumbents are soft (mechanism 2 in Section 5.3); fails if they punish and the price war costs more than it gains.
- *Steering MM.* Exploits the no-regret learners specifically: shapes the payoffs the incumbents observe so their weights converge to a spread that leaves the steering MM a profitable niche. From Deng, Schneider and Sivan.
- *Induced request exhaustion.* Section 6.2.2.
- *Predatory taker.* Section on agents above: provoke, then trade.
- *Held-out PSRO attackers.* Best responses trained against the incumbents *after* the incumbents were trained, so the incumbents never saw them.

Measured. Exploitability in bps/volume; attacker profit; and how the incumbents' collusion index Δ changes once the attack starts (does the cartel collapse?).

The hypothesis. Price-trigger punishment is a *rule*, and rules are predictable. An attacker who knows “if I undercut, they slash spreads for 5 blocks, then recover” can time its activity around that cycle. So the very mechanism that sustains rent should make the rent-earners easier to exploit.

8.3 RQ3: robust quoter and venue design

The question. You are building a market-making bot. Which design choices reduce how much you leak to attackers, and at what cost in ordinary profit? Separately: which *exchange* rules make the market more or less exploitable?

Quoter levers.

- *Update speed.* Faster reaction reduces stale quotes but burns request budget.
- *Conditioning set.* Does the quoter look only at its own fills, or at the full book with identities? More information helps until it makes you predictable.
- *Randomized offsets.* Adding noise to your quotes makes your reaction function harder to learn (the market analogue of randomized patrols in the security-game examples).
- *Opponent modelling.* Explicitly model competitors and respond to the model.
- *Inventory limits.* Tighter caps reduce cascade losses but forgo profitable fills.

Venue levers. Tick regime (coarse vs fine); rate-limit generosity; anonymous vs transparent book; priority-fee schedule; batch interval (clear every block vs every k blocks, the frequent-batch-auction idea).

Measured. For each configuration, nominal profit under ordinary conditions versus worst-case loss under: strategic attack, a liquidation cascade replay, and unseen attackers. Plotting these gives

a **profit–exploitability frontier**; a design “shifts the frontier outward” if it gets more of both. This is the efficiency–resilience trade-off the rubric asks for, stated in a trader’s units.

Benchmark policies. Random quoting; a fixed spread regardless of conditions; competitive GM.

8.4 How this fills the four course modules

Module	Where it lives in our project
Strategic modelling	The N -MM quoting game with informed takers, entrant and predator; explicit actions, information (transparent vs anonymous), pay-offs, and exact benchmarks.
Learning and adaptation	Q, Hedge and PPO incumbents; the entrant’s opponent model; compared with competitive-GM and fixed-spread policies.
Attack and defence	Attackers choose target (which MM), timing (when to strike) and type (undercut / steer / drain / predate). The defender allocates two scarce resources: inventory capacity and request budget.
Resilient design	Venue rules and quoter architecture compared under nominal and contested conditions, with the efficiency–resilience trade-off measured explicitly.

9 Part VIII: The Literature, in Plain Words

Colliard, Foucault & Lovo (RFS, forthcoming).

Put Q-learning market makers into Glosten–Milgrom. They learn to widen spreads when adverse selection rises (sensible) but never learn to undercut each other all the way to competition, so they keep a markup. The markup shrinks as more MMs enter and, surprisingly, shrinks as adverse selection rises. Their explanation is noisy feedback and limited exploration, not punishment. *Our baseline; we replicate it first.*

Cont & Xiong (Mathematical Finance 2024).

A more sophisticated continuous-time model of dealers, solved with deep multi-agent RL. The learners’ spreads end up strictly above the competitive (Nash) level: tacit collusion from learning dynamics alone. *We borrow the framing and ask a question they don’t: how exploitable are those learned policies?*

Cartea, Chang & Graumans (2025).

Using equity data *with* trader identities (rare), they show high-frequency MMs post oversized orders as signatures so other MMs recognize and avoid them, and snipe retail orders instead. Consistent with a collusive equilibrium. *On Hyperliquid identity is free; we test the same behaviour directly.*

Cartea, Chang & Penalva (2022).

Larger tick sizes make learned collusion stronger; smaller ticks make the market more competitive but slower to settle. *Motivates using Hyperliquid’s tick discontinuities as natural experiments.*

Deng, Schneider & Sivan (NeurIPS 2019).

Proves that a no-regret learner can be steered by a strategic opponent to an outcome the

opponent prefers. *Our steering attacker.*

Calvano, Calzolari, Denicolò & Pastorello (AER 2020).

The founding paper of “algorithmic collusion”: Q-learning firms setting prices learn to collude; introduces the collusion index and the impulse-response test. *We use both tools.*

Dou, Goldstein & Ji (NBER 2025).

AI *speculators* (not market makers) learn to collude by trading less aggressively, which damages price discovery; collusion only arises when prices are not already very efficient and noise is low. *Background on the price-discovery cost of collusion.*

Budish, Cramton & Shim (QJE 2015).

The latency arms race is socially wasteful; proposes frequent batch auctions. *Hyperliquid’s block structure and priority fees are a live version of this debate.*

Albers, Cucuringu, Howison & Shestopaloff (2026).

The Hyperliquid Level-4 dataset. *Our data.*

Optiver blog (2026).

Tested LLMs on their trader exam and simulations; models understood adverse selection but traded at negative expected value anyway and failed to model other participants’ reactions. *The industry statement of the skill we are trying to benchmark.*

10 Part IX: The Plan

1. **Replicate.** Two tabular Q-learners in Glosten–Milgrom. Reproduce the qualitative findings of Colliard et al.: markups fall with N and with α . If our simulator disagrees with a published result, nothing downstream is credible, so this comes first.
2. **Benchmarks.** Exact competitive and monopoly quotes on the grid; Δ ; impulse-response test to classify what the learners are doing.
3. **First RQ1 result.** $N = 2$, three toxicity levels, markout-inference entrant vs competitive benchmark.
4. **Data pipeline.** One week of BTC Level-4 data: identify MM wallets (two-sided quoting, high cancel-to-fill ratio, inventory mean-reverting to zero), estimate α from price impact, arrival rates, spread distribution.
5. **Then** RQ2 attackers, RQ3 design levers, stress grid (≈ 40 configurations $\times$ 50–100 random seeds so that we can report means, variability and tails), write-up by mid-December.

10.1 The three feedback questions, explained

1. *Stylized vs full order book.* In Glosten–Milgrom we can compute the competitive and monopoly benchmarks exactly, so Δ and exploitability are exact numbers. A full order book with queues and multiple sizes is more realistic but its competitive benchmark can only be approximated, which makes every “how collusive” claim fuzzy. We lean stylized, importing only the two Hyperliquid mechanics that change the game (request budgets, priced latency).

2. *Induced request exhaustion.* It is legal under the venue’s rules and it is a real vulnerability, but it is adversarial in spirit. We want the instructors’ view on whether to study it as an attack we *launch* or only as a threat our robust quoter must *survive*.
3. *Network size.* The scope document mentions 10–100 nodes. Our natural network is K correlated contracts with shared market makers (which also makes collusion easier, a known result about multimarket contact). We want to know whether this is required.

Common confusion

“Publish by December” does not mean “accepted by December.” A polished preprint on arXiv is realistic; peer-reviewed acceptance takes months. The plan targets the preprint.

11 Part X: What We Found, and What It Changed (added October 9)

This part was added after three days of simulation and the first Hyperliquid data. It records, in the same plain language as the rest of this document, what turned out to be true, what did not, and what that did to the plan. The numbers are in `results/REPLICATION_NOTES.md` and in the revised proposal (`paper/deliverable1_revised.tex`).

11.1 Five words you need, from the ground up

Spread A market maker posts a price at which it will buy (the bid) and a slightly higher one at which it will sell (the ask). The gap is the spread. Buy at 99.9, sell at 100.1, earn 0.2 per round trip.

Why the spread cannot be zero Some of the people hitting those quotes know something—they saw the price move elsewhere a few milliseconds ago. The maker loses on those trades (“adverse selection”). The spread must be wide enough that gains from uninformed traders cover losses to informed ones. There is a width where it exactly breaks even: the *competitive* spread.

Rent Anything the maker earns by quoting wider than break-even. It exists only because nobody is undercutting it. Same word as in economics generally: income beyond what it takes to keep you doing the job.

Undercutting Quoting a slightly better price than your competitor so that trades go to you. Two makers both selling at 100.2; one moves to 100.1 and takes every buyer, still earning 0.1 over the mid. As long as a maker can undercut and stay above break-even it has a reason to, so spreads shrink to break-even. That is why competition is supposed to kill rent.

Collusion, cartel Rent survives when makers *stop* undercutting because each has learned that undercutting starts a price war that leaves both worse off. Agreed over dinner, that is illegal collusion; reached by two reinforcement-learning agents that never communicate, it is *tacit* collusion, and it is what Colliard–Foucalt–Lovo found and we reproduced. The *cartel* is just the group of makers sustaining it. Our collusion index puts a population between break-even (0) and the single-monopolist spread (1); ours land near 0.8.

Tick Prices move in minimum steps. On Hyperliquid the step on BTC is currently \$1, on ETH \$0.10, on SOL \$0.01. The tightest possible spread is one step; there is nothing in between.

11.2 What the simulator said

1. **The cartel re-forms around an entrant.** A newcomer that reads the public tape takes most of the rent, and when the incumbents are allowed to keep learning they do not punish it or collapse; they settle into a wider three-way arrangement that includes the newcomer. The one entrant that *does* collapse the cartel—the textbook competitive quoter—earns the least, because the rent only exists while someone else quotes wide.
2. **More collusive is not more exploitable.** We expected the most collusive populations to be the easiest to exploit per unit of rent. They are not: a rational entrant takes about 70% of the rent whatever the population. We restated that question as a measurement.
3. **Identity, first attempt, failed the right way.** We modelled wallet identity as “the same toxic wallet keeps coming back”. Under that model identity was worth 26–35% of the entrant’s profit. The data then said the same wallet almost never comes back consecutively (2–5% of the time).

11.3 What the data said

1. **Knowing who just traded predicts the next fill’s toxicity**, by half to nine tenths of a basis point, on all four coins, in the archive and in our own recording. And most of that survives after accounting for what an anonymous quoter can see—whether the price has already moved against the last maker—because by the time the next order arrives, four times out of five it has not.
2. So the mechanism is **time, not wallets**: informed flow clusters in time, and a toxic wallet’s print tells you the market is in a toxic spell *before the price does*. The gap between a print and the price move that would reveal it is where identity’s value lives. We rebuilt the simulator around that (a latent toxicity regime, and a lag between a print and its mark), and identity’s value came back where the data say it should.
3. **The spread on the big coins is one tick, all the time.** So the picture in Parts II–III, where rent shows up as a wide quote and undercutting means quoting tighter, cannot be what happens on BTC, ETH or SOL under normal conditions: there is no tighter price to offer. If makers earn anything there, it is from *which fills they get*—being first in line at that one price, cancelling before the toxic flow arrives, and knowing who is active. That is the same fill-selection mechanism the identity result found, and it is the next thing to model.

11.4 What changed in the plan

The question stayed. Three edits: identity works through the regime, not the wallet; the exploitability hypothesis is now a measurement; and on tick-bound books the game is queue position rather than spread width, so the next simulator adds a queue, cancellation, and the priority fee as the price of a place in line. The data section of the paper now has numbers rather than intentions: the archive and the live feed agree, the quoted spread is a tick, and adverse selection at ten seconds is -0.33 basis points on every coin once bid–ask bounce is removed.

12 Glossary

Adverse selection Losses a market maker suffers because the traders most eager to trade with it are those who know it is mispriced.

Ask / offer Lowest price a seller currently accepts.

Basis point (bp) One hundredth of a percent, 0.0001.

Best response The most profitable strategy against a fixed set of opponent strategies.

Bid Highest price a buyer currently pays.

Block One discrete time step in which the exchange processes a batch of orders.

Collusion index Δ Fraction of the way from competitive profit to monopoly profit.

Discount factor γ Weight placed on next period's reward relative to this period's.

Effective / realized spread What a taker paid relative to mid / what the maker kept after the subsequent price move.

Episode One game of T blocks with a fixed true value V .

Exploitability How much a frozen policy loses to its best-responding adversary relative to its nominal environment (L_{BR}); the market is general-sum, so incumbent loss, not attacker gain, is the statistic.

Fundamental V The true value of the asset; unknown during the episode, revealed at the end.

Glosten–Milgrom The basic model of a market maker facing a mix of informed and uninformed traders.

Grim trigger / price trigger A strategy that cooperates until someone defects, then punishes.

Hazard λ Per-block probability of an event (here, a liquidation).

HLP Hyperliquid's protocol-run market-making and liquidation vault; its strategies are undisclosed, so we use an HLP-inspired calibrated benchmark, not HLP itself.

Impulse-response test Force one agent to deviate and watch whether the others punish.

Inventory I Net position a market maker holds.

Limit / market order Resting order at a chosen price / immediate order at the best available price.

Liquidation Forced closure of a leveraged position that has run out of margin.

Maker / taker Provider / remover of liquidity.

Markout Price movement after a fill, used to measure adverse selection.

Mid-price m Average of best bid and best ask.

Nash equilibrium Strategy profile from which no single player wants to deviate.

No-regret learner Algorithm guaranteed to do as well as the best fixed action in hindsight.

Perpetual future Leveraged derivative that tracks an asset with no expiry.

Price–time priority Better price first; among equal prices, earlier order first.

Priority fee Payment that reduces an order’s latency on Hyperliquid.

PSRO Iteratively training best responses to build a pool of unseen attackers.

Q-learning Table-based RL that learns the value of each action in each state.

Rent Profit above what competition would allow.

Request budget B Number of order actions an address may send; earned by traded volume on Hyperliquid.

Spread Best ask minus best bid.

Tacit collusion Supra-competitive outcomes sustained by punishment threats without communication.

Tape The public record of quotes and trades.

Wallet identity A persistent pseudonymous address attached to every committed order on Hyperliquid; visible after the block commits, not before.

Tick size θ Minimum price increment.

Toxicity α Fraction of order flow that is informed.